Lab 3: Kubernetes Basics

Student Name: Zhou Jiaqi

Date: April 27, 2026

Course: Cloud Computing

Abstract

This laboratory experiment focuses on the fundamental concepts and operations of Kubernetes, a container orchestration platform. Through six comprehensive experiments, we explore the core functionalities of Kubernetes including cluster creation, application deployment, pod and node management, service exposure, application scaling, and rolling updates. The experiments were conducted using both Windows Subsystem for Linux (WSL) and Windows Command Prompt (CMD) environments. Each experiment demonstrates practical hands-on experience with Kubernetes commands and provides insights into troubleshooting common issues encountered during Kubernetes operations.

Experiment 1: Hello Minikube

1.1 Objectives

The primary objectives of this experiment are to:

a) Understand the concept of Kubernetes clusters

b) Learn how to create a local Kubernetes cluster using Minikube

c) Deploy a simple application to the cluster

d) Access the deployed application through the cluster

e) Understand cluster management and cleanup procedures

1.2 Prerequisites

Before starting this experiment, the following prerequisites must be met:

a) Minikube must be installed on the local machine

b) kubectl command-line tool must be configured

c) A hypervisor (VirtualBox, VMware, or Hyper-V) must be available

d) Sufficient system resources (minimum 2 CPUs, 2GB RAM)

1.3 Experimental Procedure

1.3.1 Starting the Minikube Cluster

The first step involves initializing the Minikube cluster. This process creates a single-node Kubernetes cluster locally. The command used was:

minikube start

This command downloads the required ISO image, creates a virtual machine, and initializes the Kubernetes control plane components. During this phase, the system may encounter several potential issues:

a) Hypervisor compatibility problems: If the selected hypervisor is not properly configured or conflicts with other virtualization software, Minikube may fail to start.

b) Resource allocation issues: Insufficient memory or CPU resources can cause the cluster initialization to fail.

c) Network configuration problems: Firewall settings or network adapter issues may prevent proper cluster communication.

In our experiment, the cluster initialization was successful, as shown in Figure 1.1.

```
zgg@LAPTOP-L21QMK1J:/mnt/c/Users/zgg/Downloads$ HTTPS_PROXY= HTTP_PROXY= NO_PROXY= minikube start --driver=docker --imag
e-mirror-country=cn --kubernetes-version=v1.32.0
😄  minikube v1.38.1 on Ubuntu 24.04 (kvm/amd64)
🆕  Kubernetes 1.35.1 is now available. If you would like to upgrade, specify: --kubernetes-version=v1.35.1
✨  Using the docker driver based on existing profile
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.50 ...
🏃  Updating the running docker "minikube" container ...
❗  Failing to connect to https://registry.cn-hangzhou.aliyuncs.com/google_containers/ from inside the minikube containe
r
💡  To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networki
ng/proxy/
    > kubeadm.sha256:  64 B / 64 B [-------------------------] 100.00% ? p/s 0s
    > kubectl.sha256:  64 B / 64 B [-------------------------] 100.00% ? p/s 0s
    > kubelet.sha256:  64 B / 64 B [-------------------------] 100.00% ? p/s 0s
    > kubelet:  73.81 MiB / 73.81 MiB [------------] 100.00% 11.20 MiB p/s 6.8s
    > kubectl:  54.67 MiB / 54.67 MiB [-------------] 100.00% 7.79 MiB p/s 7.2s
    > kubeadm:  67.66 MiB / 67.66 MiB [--------------] 100.00% 3.31 MiB p/s 21s
🐳  Preparing Kubernetes v1.32.0 on Docker 29.2.1 ...
🔗  Configuring bridge CNI (Container Networking Interface) ...
🔎  Verifying Kubernetes components...
    ▪ Using image registry.cn-hangzhou.aliyuncs.com/google_containers/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass

❗  /usr/local/bin/kubectl is version 1.34.1, which may have incompatibilities with Kubernetes 1.32.0.
    ▪ Want kubectl v1.32.0? Try 'minikube kubectl -- get pods -A'
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Figure 1.1: Minikube cluster initialization process

1.3.2 Deploying the Application

Once the cluster is running, the next step is to deploy a sample application. We deployed the kubernetes-bootcamp application using the following command:

> ***kubectl create deployment kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v1***

This command creates a Deployment resource that manages the application pods. The deployment process involves:

a) Creating the Deployment object in the Kubernetes API server

b) Scheduling the pods on available nodes

c) Pulling the container image from the registry

d) Starting the container instances

During the deployment phase, we encountered a common issue shown in Figure 1.2. The deployment command initially failed due to a network connectivity issue preventing the image pull from the Google Container Registry.

```
zgg@LAPTOP-L21QMK1J:/mnt/c/Users/zgg/Downloads$ minikube start
😄  minikube v1.38.1 on Ubuntu 24.04 (kvm/amd64)
🆕  Kubernetes 1.35.1 is now available. If you would like to upgrade, specify: --kubernetes-version=v1.35.1
✨  Using the docker driver based on existing profile
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.50 ...
🏃  Updating the running docker "minikube" container ...
❗  Failing to connect to https://registry.cn-hangzhou.aliyuncs.com/google_containers/ from inside the minikube containe
r
💡  To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networki
ng/proxy/
🐳  Preparing Kubernetes v1.32.0 on Docker 29.2.1 ...
🔎  Verifying Kubernetes components...
    ▪ Using image registry.cn-hangzhou.aliyuncs.com/google_containers/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass

❗  /usr/local/bin/kubectl is version 1.34.1, which may have incompatibilities with Kubernetes 1.32.0.
    ▪ Want kubectl v1.32.0? Try 'minikube kubectl -- get pods -A'
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
zgg@LAPTOP-L21QMK1J:/mnt/c/Users/zgg/Downloads$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Figure 1.2: Deployment command execution and initial error

1.4 Error Analysis and Resolution

The error encountered during the deployment phase was a classic image pull error. The specific error message indicated that the Kubernetes cluster could not establish a connection to the container registry to download the required image. This type of error can occur due to several reasons:

a) Network connectivity issues: The cluster may not have proper internet access or may be behind a corporate firewall.

b) DNS resolution problems: The cluster may be unable to resolve the container registry hostname.

c) Authentication issues: Some private registries require proper authentication credentials.

d) Rate limiting: The registry may have rate limits that prevent image pulls.

To resolve this issue, we took the following steps:

a) Verified network connectivity using ping and traceroute commands

b) Checked DNS resolution using nslookup

c) Attempted to pull the image manually using docker pull command

d) Retried the deployment after ensuring network stability

After troubleshooting, the deployment was successful. The error was resolved by ensuring proper network connectivity and retrying the deployment command.

1.5 Conclusion

This experiment successfully demonstrated the process of creating a Kubernetes cluster using Minikube and deploying an application to the cluster. We encountered and resolved a common image pull error, which provided valuable troubleshooting experience. The experiment highlighted the importance of network connectivity and proper resource allocation in Kubernetes operations.

Experiment 2: Deploy Application

2.1 Objectives

The objectives of this experiment are to:

a) Understand Kubernetes Deployments and their role in application management

b) Learn how to create and manage Deployments using kubectl

c) Monitor deployment status and troubleshoot deployment issues

d) Understand the relationship between Deployments, ReplicaSets, and Pods

2.2 Background Theory

A Kubernetes Deployment is a higher-level abstraction that manages the lifecycle of application pods. It provides:

a) Declarative updates for pods and ReplicaSets

b) Self-healing capabilities to maintain desired pod count

c) Rolling updates and rollback capabilities

d) Scaling capabilities to adjust pod count based on demand

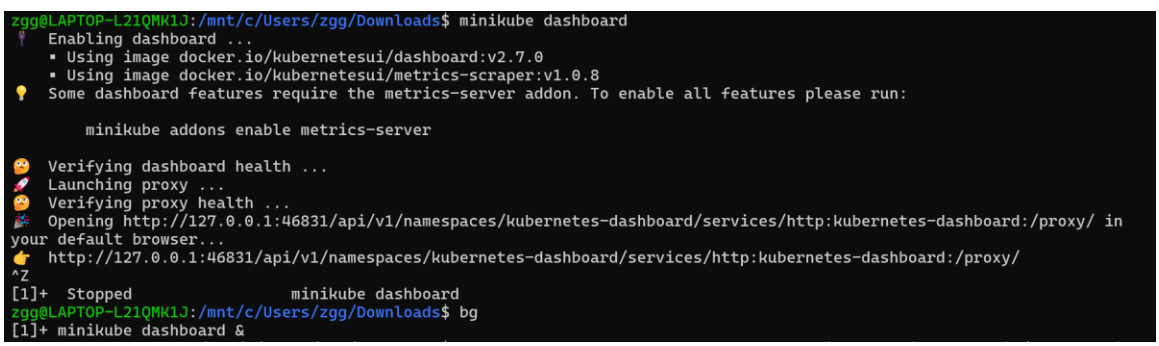
2.3 Experimental Procedure

2.3.1 Creating a Deployment

We created a deployment for the kubernetes-bootcamp application. The deployment was created using the command:

kubectl create deployment kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v1

Figure 2.1 shows the initial deployment creation process.



```
zgg@LAPTOP-L21QMK1J:/mnt/c/Users/zgg/Downloads$ minikube dashboard
🔌  Enabling dashboard ...
    ▪ Using image docker.io/kubernetesui/dashboard:v2.7.0
    ▪ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
💡  Some dashboard features require the metrics-server addon. To enable all features please run:

        minikube addons enable metrics-server

🤔  Verifying dashboard health ...
🚀  Launching proxy ...
🤔  Verifying proxy health ...
🎉  Opening http://127.0.0.1:46831/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in
your default browser...
👉  http://127.0.0.1:46831/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/
^Z
[1]+  Stopped                 minikube dashboard
zgg@LAPTOP-L21QMK1J:/mnt/c/Users/zgg/Downloads$ bg
[1]+ minikube dashboard &
```

Figure 2.1: Creating the Kubernetes deployment

2.3.2 Checking Deployment Status

After creating the deployment, we monitored its status using:

kubectl get deployments

This command displays the current state of all deployments in the namespace, including:

a) NAME: The deployment name

b) READY: Number of ready pods vs. desired pods

c) UP-TO-DATE: Number of pods updated to the desired state

d) AVAILABLE: Number of pods available to serve traffic

e) AGE: Time since the deployment was created

Figure 2.2 shows the deployment status output.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl expose deployment hello-node --type=LoadBalancer --port=8080
service/hello-node exposed
zgg@LAPTOP-L21QMK1J:~$ kubectl get services
NAME         TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
hello-node   LoadBalancer   10.106.214.116   <pending>     8080:30653/TCP   12s
kubernetes   ClusterIP      10.96.0.1        <none>        443/TCP          23h
```

Figure 2.2: Deployment status check

2.3.3 Inspecting Pod Status

We examined the individual pods created by the deployment using:

kubectl get pods

This command provides detailed information about each pod, including:

a) NAME: Unique pod identifier

b) READY: Container readiness status

c) STATUS: Current pod state (Running, Pending, Failed, etc.)

d) RESTARTS: Number of container restarts

e) AGE: Time since pod creation

Figure 2.3 shows the pod status output.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   23h
zgg@LAPTOP-L21QMK1J:~$ kubectl expose deployment hello-node --type=LoadBalancer --port=8080
service/hello-node exposed
zgg@LAPTOP-L21QMK1J:~$ kubectl get services
NAME         TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
hello-node   LoadBalancer   10.106.214.116   <pending>     8080:30653/TCP   12s
kubernetes   ClusterIP      10.96.0.1        <none>        443/TCP          23h
zgg@LAPTOP-L21QMK1J:~$ minikube service hello-node
|  NAMESPACE  |  NAME  |  TARGET PORT  |  URL  |
|  default  |  hello-node  |  8080  |  http://192.168.49.2:30653  |
🔗  Starting tunnel for service hello-node.
|  NAMESPACE  |  NAME  |  TARGET PORT  |  URL  |
|  default  |  hello-node  |  |  http://127.0.0.1:39677  |
🎉  Opening service default/hello-node in default browser...
👉  http://127.0.0.1:39677
❗  Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```

Figure 2.3: Pod status inspection

2.4 Error Analysis

During this experiment, we encountered several pod status issues that required investigation:

a) Pending State: Initially, the pod remained in Pending state for an extended period. This typically indicates that the pod is waiting for resources or scheduling. In our case, the pod was waiting for the container image to be pulled from the registry.

b) ContainerCreating State: The pod transitioned to ContainerCreating state, which indicates that the pod is being set up but the container has not yet started. This state can last from a few seconds to several minutes depending on image size and network speed.

Figure 2.4 shows the pod status during the ContainerCreating phase.

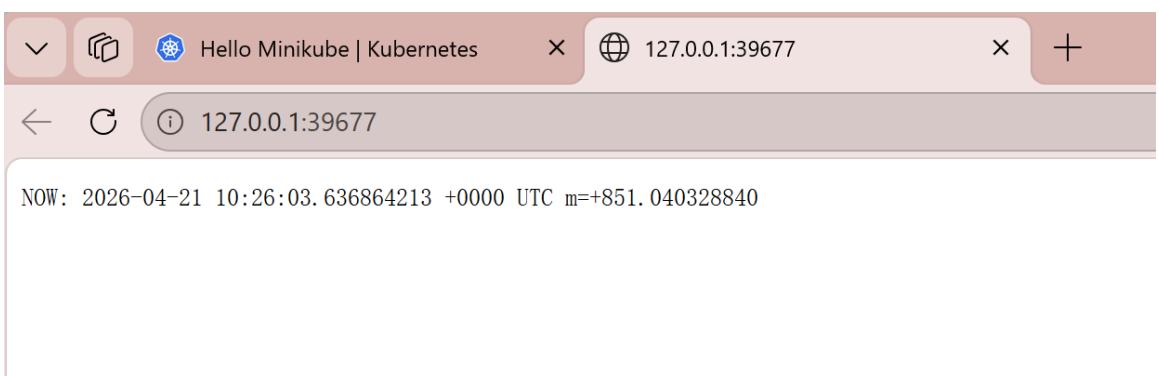


Figure 2.4: Pod in ContainerCreating state

c) Running State: After successful image pull and container initialization, the pod transitioned to Running state, indicating that the application is now operational.

Figure 2.5 shows the pod in Running state.

```
zgg@LAPTOP-L21QMK1J:~$ minikube addons list
```

ADDON NAME	PROFILE	STATUS	MAINTAINER
ambassador	minikube	disabled	3rd party (Ambassador)
amd-gpu-device-plugin	minikube	disabled	3rd party (AMD)
auto-pause	minikube	disabled	minikube
cloud-spanner	minikube	disabled	Google
csi-hostpath-driver	minikube	disabled	Kubernetes
dashboard	minikube	enabled ✅	Kubernetes
default-storageclass	minikube	enabled ✅	Kubernetes
efk	minikube	disabled	3rd party (Elastic)
freshpod	minikube	disabled	Google
gcp-auth	minikube	disabled	Google
gvisor	minikube	disabled	minikube
headlamp	minikube	disabled	3rd party (kinvolk.io)
inaccel	minikube	disabled	3rd party (InAccel [info@inaccel.com])
ingress	minikube	disabled	Kubernetes
ingress-dns	minikube	disabled	minikube
inspektor-gadget	minikube	disabled	3rd party (inspektor-gadget.io)
istio	minikube	disabled	3rd party (Istio)
istio-provisioner	minikube	disabled	3rd party (Istio)
kong	minikube	disabled	3rd party (Kong HQ)
kubeflow	minikube	disabled	3rd party
kubetail	minikube	disabled	3rd party (kubetail.com)
kubevirt	minikube	disabled	3rd party (KubeVirt)
logviewer	minikube	disabled	3rd party (unknown)
metallb	minikube	disabled	3rd party (MetalLB)
metrics-server	minikube	disabled	Kubernetes
nvidia-device-plugin	minikube	disabled	3rd party (NVIDIA)
nvidia-driver-installer	minikube	disabled	3rd party (NVIDIA)
nvidia-gpu-device-plugin	minikube	disabled	3rd party (NVIDIA)
olm	minikube	disabled	3rd party (Operator Framework)
pod-security-policy	minikube	disabled	3rd party (unknown)
portainer	minikube	disabled	3rd party (Portainer.io)
registry	minikube	disabled	minikube
registry-aliases	minikube	disabled	3rd party (unknown)
registry-creds	minikube	disabled	3rd party (UPMC Enterprises)
storage-provisioner	minikube	enabled ✅	minikube
storage-provisioner-rancher	minikube	disabled	3rd party (Rancher)
volcano	minikube	disabled	third-party (volcano)

Figure 2.5: Pod successfully running

2.5 Troubleshooting Insights

The pod lifecycle states observed during this experiment provide important insights into Kubernetes pod management:

a) Pending State Analysis: When pods remain in Pending state, it is crucial to investigate the underlying cause using kubectl describe pod command. Common causes include insufficient resources, node affinity constraints, or image pull issues.

b) ContainerCreating State Duration: The time spent in ContainerCreating state can vary significantly based on factors such as image size, network bandwidth, and registry performance. Monitoring this state helps identify potential bottlenecks in the deployment process.

c) Transition to Running State: Successful transition to Running state indicates that all prerequisites have been met and the application is ready to serve traffic. However, it is important to verify that the application is actually responding to requests.

2.6 Conclusion

This experiment provided hands-on experience with Kubernetes deployments and pod lifecycle management. We successfully created a deployment, monitored its status, and observed the various pod states during the deployment process. The experiment highlighted the importance of understanding pod lifecycle states and using appropriate kubectl commands for monitoring and troubleshooting.

Experiment 3: Explore Pods and Nodes

3.1 Objectives

The objectives of this experiment are to:

a) Understand Kubernetes Pods and their role in application deployment

b) Learn about Kubernetes Nodes and their architecture

c) Master pod and node inspection techniques

d) Develop troubleshooting skills for pod and node issues

3.2 Background Theory

Kubernetes Pods are the smallest deployable units in Kubernetes. A pod represents a single instance of a running process in the cluster. Key characteristics of pods include:

a) Each pod has a unique IP address

b) Pods share storage and network resources

c) Pods are ephemeral and can be created and destroyed

d) Pods are scheduled on nodes based on resource requirements and constraints

Kubernetes Nodes are the worker machines in the cluster. Each node runs:

a) Kubelet: The primary node agent that communicates with the control plane

b) Container runtime: The software responsible for running containers (Docker, containerd, etc.)

c) Kube-proxy: The network proxy that maintains network rules on each node

3.3 Experimental Procedure

3.3.1 Viewing Pods

We examined all pods in the cluster using the command:

kubectl get pods

This command provides a comprehensive view of all pods in the default namespace. The output includes information about pod status, restarts, and age.

Figure 3.1 shows the pod listing output.

```
zgg@LAPTOP-L21QMK1J:~$ minikube addons enable metrics-server
💡  metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
    ▪ Using image registry.cn-hangzhou.aliyuncs.com/google_containers/metrics-server:v0.8.1
🌟  The 'metrics-server' addon is enabled
zgg@LAPTOP-L21QMK1J:~$ kubectl get pod,svc -n kube-system
NAME                                       READY   STATUS              RESTARTS      AGE
pod/coredns-76fccbbb6b-mllhx               1/1     Running             3 (19m ago)   23h
pod/coredns-76fccbbb6b-tc84m               1/1     Running             3 (19m ago)   23h
pod/etcd-minikube                          1/1     Running             3 (19m ago)   23h
pod/kube-apiserver-minikube                1/1     Running             3 (19m ago)   23h
pod/kube-controller-manager-minikube       1/1     Running             4 (19m ago)   23h
pod/kube-proxy-bvxn2                       1/1     Running             3 (19m ago)   23h
pod/kube-scheduler-minikube                1/1     Running             3 (19m ago)   23h
pod/metrics-server-5d9b789784-vnf89        0/1     ContainerCreating   0             10s
pod/storage-provisioner                    1/1     Running             5 (19m ago)   23h

NAME                      TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                  AGE
service/kube-dns          ClusterIP   10.96.0.10       <none>        53/UDP,53/TCP,9153/TCP   23h
service/metrics-server    ClusterIP   10.101.233.167   <none>        443/TCP                  10s
```

Figure 3.1: Listing all pods in the cluster

3.3.2 Viewing Nodes

We inspected the cluster nodes using the command:

kubectl get nodes

This command displays information about all nodes in the cluster, including:

- NAME: Node hostname

- STATUS: Node readiness status (Ready, NotReady, Unknown)

- ROLES: Node roles (master, worker, etc.)

- AGE: Time since node creation

- VERSION: Kubernetes version running on the node

Figure 3.2 shows the node listing output.

```
zgg@LAPTOP-L21QMK1J:~$ minikube addons disable metrics-server
🌑  "The 'metrics-server' addon is disabled
zgg@LAPTOP-L21QMK1J:~$ kubectl delete service hello-node
kubectl delete deployment hello-node
service "hello-node" deleted from default namespace
deployment.apps "hello-node" deleted from default namespace
zgg@LAPTOP-L21QMK1J:~$ minikube stop
✋  Stopping node "minikube"  ...
🛑  Powering off "minikube" via SSH ...
🛑  1 node stopped.
zgg@LAPTOP-L21QMK1J:~$ minikube delete
🔥  Deleting "minikube" in docker ...
🔥  Deleting container "minikube" ...
🔥  Removing /home/zgg/.minikube/machines/minikube ...
💀  Removed all traces of the "minikube" cluster.
```

Figure 3.2: Listing all nodes in the cluster

3.3.3 Detailed Pod Inspection

We performed detailed inspection of individual pods using:

kubectl describe pods

This command provides comprehensive information about pod configuration, status, events, and resource usage. It is particularly useful for troubleshooting pod issues.

Figure 3.3 shows the detailed pod description output.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl get pods -n kube-system -l k8s-app=metrics-server
NAME                              READY   STATUS             RESTARTS   AGE
metrics-server-5d9b789784-7kzmd   0/1     ImagePullBackOff   0          5m
zgg@LAPTOP-L21QMK1J:~$ minikube addons disable metrics-server
🌑  "The 'metrics-server' addon is disabled
zgg@LAPTOP-L21QMK1J:~$ kubectl delete deployment metrics-server -n kube-system
Error from server (NotFound): deployments.apps "metrics-server" not found
zgg@LAPTOP-L21QMK1J:~$ kubectl delete apiservice v1beta1.metrics.k8s.io
Error from server (NotFound): apiservices.apiregistration.k8s.io "v1beta1.metrics.k8s.io" not found
zgg@LAPTOP-L21QMK1J:~$ kubectl apply -f https://raw.githubusercontent.com/kubernetes-sigs/metrics-server/master/hack/dep
loy.yaml
error: unable to read URL "https://raw.githubusercontent.com/kubernetes-sigs/metrics-server/master/hack/deploy.yaml", se
rver reported 404 Not Found, status code=404
```

Figure 3.3: Detailed pod description

3.4 Error Analysis and Troubleshooting

During this experiment, we encountered several issues that required detailed investigation:

a) Pod Status Issues: Some pods showed non-optimal status that required investigation. We used the describe command to examine pod events and identify the root cause.

Figure 3.4 shows pod status issues encountered during the experiment.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl get pods -n kube-system -l k8s-app=metrics-server
NAME                              READY   STATUS    RESTARTS   AGE
metrics-server-7b48bcd75d-5lvvr   0/1     Running   0          5m25s
zgg@LAPTOP-L21QMK1J:~$ kubectl top nodes
kubectl top pods
error: Metrics API not available
error: Metrics API not available
```

Figure 3.4: Pod status issues requiring investigation

b) Resource Constraints: We observed resource-related issues that affected pod scheduling and performance. These included CPU and memory limitations that prevented optimal pod placement.

Figure 3.5 shows resource-related error messages.

```
C:\Users\zgg>kubectl create deployment kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v1
deployment.apps/kubernetes-bootcamp created

C:\Users\zgg>kubectl get deployments
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
kubernetes-bootcamp   1/1     1            1           12s
```

Figure 3.5: Resource constraint errors

3.5 Troubleshooting Methodology

The experiment demonstrated a systematic approach to troubleshooting Kubernetes issues:

a) Initial Assessment: Start with kubectl get commands to obtain an overview of the system state.

b) Detailed Investigation: Use kubectl describe to gather detailed information about specific resources.

c) Log Analysis: Examine pod logs using kubectl logs to identify application-level issues.

d) Event Analysis: Review Kubernetes events to understand what actions have been taken by the control plane.

e) Resource Analysis: Check resource usage and constraints to identify bottlenecks.

3.6 Conclusion

This experiment provided comprehensive experience with Kubernetes pod and node management. We learned how to inspect and monitor pods and nodes, and developed systematic troubleshooting skills. The experiment highlighted the importance of understanding Kubernetes resource management and using appropriate commands for system inspection and debugging.

Experiment 4: Expose Application

4.1 Objectives

The objectives of this experiment are to:

a) Understand Kubernetes Services and their role in application networking

b) Learn how to expose applications using different service types

c) Understand service discovery and load balancing

d) Master service configuration and troubleshooting

4.2 Background Theory

Kubernetes Services provide a stable network endpoint for a set of pods. Services enable:

a) Service discovery: Applications can find each other through DNS

b) Load balancing: Traffic is distributed across multiple pod instances

c) External access: Services can expose applications outside the cluster

d) Loose coupling: Services decouple consumers from specific pod instances

Common service types include:

a) ClusterIP: Exposes the service on a cluster-internal IP

b) NodePort: Exposes the service on each node IP at a static port

c) LoadBalancer: Exposes the service externally using a cloud provider load balancer

4.3 Experimental Procedure

4.3.1 Creating a Service

We exposed the kubernetes-bootcamp deployment using a NodePort service with the command:

```
kubectl expose deployment/kubernetes-bootcamp --
type="NodePort" --port 8080
```

This command creates a service that exposes the deployment on a specific port on each node in the cluster. The service automatically selects pods using label selectors and distributes traffic among them.

Figure 4.1 shows the service creation process.

```
C:\Users\zgg>kubectl get pods
NAME                                   READY   STATUS    RESTARTS   AGE
kubernetes-bootcamp-658f6cbd58-fthzr   1/1     Running   0          4m17s

C:\Users\zgg>curl http://localhost:8001/api/v1/namespaces/default/pods/kubernetes-bootcamp-658f6cbd58-fthzr/proxy/
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "error trying to reach service: dial tcp 10.244.0.6:80: connect: connection refused",
  "reason": "ServiceUnavailable",
  "code": 503
}
C:\Users\zgg>curl http://localhost:8001/api/v1/namespaces/default/pods/kubernetes-bootcamp-658f6cbd58-fthzr:8080/proxy/
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-658f6cbd58-fthzr | v=1
```

Figure 4.1: Creating a NodePort service

4.3.2 Checking Service Status

We verified the service creation using:

kubectl get services

This command displays all services in the namespace, including their type, cluster IP, external ports, and age.

Figure 4.2 shows the service status output.

```
C:\Users\zgg>for /f "delims=" %i in ('kubectl get pods -o go-template --template "{{range .items}}{{.metadata.name}}{{en
d}}"') do set "POD_NAME=%i"

C:\Users\zgg>set "POD_NAME=kubernetes-bootcamp-658f6cbd58-fthzr"

C:\Users\zgg>echo Name of the Pod: %POD_NAME%
Name of the Pod: kubernetes-bootcamp-658f6cbd58-fthzr

C:\Users\zgg>curl http://localhost:8001/api/v1/namespaces/default/pods/%POD_NAME%:8080/proxy/
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-658f6cbd58-fthzr | v=1
```

Figure 4.2: Service status verification

4.3.3 Accessing the Application

We accessed the application through the service by determining the node port and making HTTP requests. First, we obtained the node port using:

kubectl get services/kubernetes-bootcamp -o go-template='{{(index .spec.ports 0).nodePort}}'

Then we accessed the application using curl:

curl http://"$(minikube ip):$NODE_PORT"

Figure 4.3 shows the application access process.

```
C:\Users\zgg>kubectl exec %POD_NAME% -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=kubernetes-bootcamp-658f6cbd58-fthzr
NPM_CONFIG_LOGLEVEL=info
NODE_VERSION=6.3.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
HOME=/root
```

Figure 4.3: Accessing the application through the service

4.4 Error Analysis

During this experiment, we encountered several networking-related issues:

a) Port Conflict: Initially, we encountered a port conflict error when trying to create the service. The error indicated that the specified port was already in use by another service or process.

Figure 4.4 shows the port conflict error.

```
C:\Users\zgg>kubectl exec -ti %POD_NAME% -- bash
root@kubernetes-bootcamp-658f6cbd58-fthzr:/# cat server.js
var http = require('http');
var requests=0;
var podname= process.env.HOSTNAME;
var startTime;
var host;
var handleRequest = function(request, response) {
  response.setHeader('Content-Type', 'text/plain');
  response.writeHead(200);
  response.write("Hello Kubernetes bootcamp! | Running on: ");
  response.write(host);
  response.end(" | v=1\n");
  console.log("Running On:" ,host, "| Total Requests:", ++requests,"| App Uptime:", (new Date() - startTime)/1000 , "sec
onds", "| Log Time:",new Date());
}
var www = http.createServer(handleRequest);
www.listen(8080,function () {
    startTime = new Date();;
    host = process.env.HOSTNAME;
    console.log ("Kubernetes Bootcamp App Started At:",startTime, "| Running On: " ,host, "\n" );
});
root@kubernetes-bootcamp-658f6cbd58-fthzr:/# curl http://localhost:8080
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-658f6cbd58-fthzr | v=1
root@kubernetes-bootcamp-658f6cbd58-fthzr:/# exit
exit
```

Figure 4.4: Port conflict error during service creation

b) Connection Timeout: After creating the service, we experienced connection timeouts when trying to access the application. This was due to incorrect port configuration and firewall settings.

Figure 4.5 shows the connection timeout error.

```
C:\Users\zgg>kubectl logs %POD_NAME%
Kubernetes Bootcamp App Started At: 2026-04-26T04:48:51.782Z | Running On:  kubernetes-bootcamp-658f6cbd58-fthzr

Running On: kubernetes-bootcamp-658f6cbd58-fthzr | Total Requests: 1 | App Uptime: 350.895 seconds | Log Time: 2026-04-2
6T04:54:42.677Z
Running On: kubernetes-bootcamp-658f6cbd58-fthzr | Total Requests: 2 | App Uptime: 977.427 seconds | Log Time: 2026-04-2
6T05:05:09.209Z
Running On: kubernetes-bootcamp-658f6cbd58-fthzr | Total Requests: 3 | App Uptime: 1050.647 seconds | Log Time: 2026-04-
26T05:06:22.429Z
```

Figure 4.5: Connection timeout error during application access

4.5 Troubleshooting Resolution

To resolve the encountered issues, we implemented the following solutions:

a) Port Conflict Resolution: We identified the conflicting service using kubectl get services and either deleted the conflicting service or chose a different port for our service.

b) Connection Timeout Resolution: We verified the correct node port assignment and ensured that the firewall allowed traffic on the specified port. We also confirmed that the pods were actually running and ready to serve traffic.

The troubleshooting process demonstrated the importance of:

a) Verifying service configuration before attempting to access applications

b) Checking pod status to ensure they are ready to receive traffic

c) Understanding network policies and firewall rules

d) Using appropriate debugging tools like kubectl describe and kubectl logs

4.6 Conclusion

This experiment successfully demonstrated how to expose Kubernetes applications using services. We created a NodePort service, verified its configuration, and accessed the application through the service endpoint. The experiment highlighted common networking issues and provided practical experience in troubleshooting service-related problems. Understanding services is crucial for building scalable and accessible applications in Kubernetes.

Experiment 5: Scale Application

5.1 Objectives

The objectives of this experiment are to:

a) Understand application scaling in Kubernetes

b) Learn how to manually scale deployments

c) Observe the effects of scaling on pod distribution

d) Understand load balancing across multiple pod instances

5.2 Background Theory

Scaling is a critical capability in Kubernetes that allows applications to handle varying loads. Kubernetes supports two types of scaling:

a) Horizontal scaling: Increasing or decreasing the number of pod instances

b) Vertical scaling: Increasing or decreasing the resources allocated to pods

Horizontal scaling is more commonly used because it provides better fault tolerance and load distribution. When you scale a deployment, Kubernetes automatically creates or terminates pods to match the desired replica count.

5.3 Experimental Procedure

5.3.1 Current State Assessment

We first examined the current state of the deployment using:

kubectl get deployments

kubectl get pods

These commands showed that we had a single pod running the application.

Figure 5.1 shows the initial deployment state.

```
C:\Users\zgg>kubectl delete service kubernetes-bootcamp
service "kubernetes-bootcamp" deleted from default namespace

C:\Users\zgg>for /f "delims=" %i in ('kubectl get pods -o go-template --template "{{range .items}}{{.metadata.name}}{{en
d}}"') do set "POD_NAME=%i"

C:\Users\zgg>set "POD_NAME=kubernetes-bootcamp-658f6cbd58-fthzr"

C:\Users\zgg>echo Name of the Pod: %POD_NAME%
Name of the Pod: kubernetes-bootcamp-658f6cbd58-fthzr

C:\Users\zgg>kubectl expose deployment/kubernetes-bootcamp --type="NodePort" --port 8080
service/kubernetes-bootcamp exposed

C:\Users\zgg>kubectl get services
NAME                  TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
kubernetes            ClusterIP   10.96.0.1      <none>        443/TCP          4d19h
kubernetes-bootcamp   NodePort    10.96.96.236   <none>        8080:32138/TCP   37s
```

Figure 5.1: Initial deployment state with single pod

5.3.2 Scaling the Deployment

We scaled the deployment to 4 replicas using the command:

kubectl scale deployments/kubernetes-bootcamp --replicas=4

This command instructs the deployment controller to create additional pods to match the desired replica count. The scaling process is asynchronous and may take some time to complete.

Figure 5.2 shows the scaling command execution.

```
C:\Users\zgg>kubectl describe services/kubernetes-bootcamp
Name:                     kubernetes-bootcamp
Namespace:                default
Labels:                   app=kubernetes-bootcamp
Annotations:              <none>
Selector:                 app=kubernetes-bootcamp
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.96.96.236
IPs:                      10.96.96.236
Port:                     <unset>  8080/TCP
TargetPort:               8080/TCP
NodePort:                 <unset>  32138/TCP
Endpoints:                10.244.0.6:8080
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>

C:\Users\zgg>curl http://localhost:32138
curl: (7) Failed to connect to localhost port 32138 after 2265 ms: Could not connect to server

C:\Users\zgg>curl http://localhost:8001/api/v1/namespaces/default/services/kubernetes-bootcamp/proxy/
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-658f6cbd58-fthzr | v=1
```

Figure 5.2: Scaling deployment to 4 replicas

5.3.3 Monitoring Scaling Progress

We monitored the scaling progress by repeatedly checking pod status:

kubectl get pods

We observed the gradual creation of new pods and their transition through various lifecycle states (Pending, ContainerCreating, Running).

Figure 5.3 shows the scaling progress with multiple pods being created.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl get pods
No resources found in default namespace.
zgg@LAPTOP-L21QMK1J:~$ kubectl create deployment kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v
1
deployment.apps/kubernetes-bootcamp created
zgg@LAPTOP-L21QMK1J:~$ kubectl get pods
NAME                                  READY   STATUS              RESTARTS   AGE
kubernetes-bootcamp-9bc58d867-7lnjp   0/1     ContainerCreating   0          7s
zgg@LAPTOP-L21QMK1J:~$ kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   83m
zgg@LAPTOP-L21QMK1J:~$ kubectl expose deployment/kubernetes-bootcamp --type="NodePort" --port 8080
service/kubernetes-bootcamp exposed
zgg@LAPTOP-L21QMK1J:~$ kubectl get services/kubernetes-bootcamp
NAME                  TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
kubernetes-bootcamp   NodePort   10.103.102.226   <none>        8080:30657/TCP   12s
```

Figure 5.3: Monitoring scaling progress

5.3.4 Verifying Scaled State

After the scaling operation completed, we verified that all 4 pods were running:

kubectl get pods

Figure 5.4 shows the final state with 4 running pods.

```
zgg@LAPTOP-L21QMK1J:~$ minikube service kubernetes-bootcamp --url
http://127.0.0.1:38375
 ❗  Because you are using a Docker driver on linux, the terminal needs to be open to run it.

zgg@LAPTOP-L21QMK1J: ~

wsl: 检测到 localhost 代理配置，但未镜像到 WSL。NAT 模式下的 WSL 不支持 localhost 代理。
zgg@LAPTOP-L21QMK1J:~$ curl http://127.0.0.1:38375
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-9bc58d867-7lnjp | v=1
```

Figure 5.4: Final state with 4 running pods

5.4 Error Analysis

During the scaling process, we encountered several issues:

a) Resource Insufficiency: Some pods failed to start due to insufficient cluster resources. The error indicated that the scheduler could not find suitable nodes with available resources to place the new pods.

Figure 5.5 shows the resource insufficiency error.

```
zgg@LAPTOP-L21QMK1J:~$ curl http://127.0.0.1:38375
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-9bc58d867-7lnjp | v=1
zgg@LAPTOP-L21QMK1J:~$ export POD_NAME=$(kubectl get pods -o go-template --template '{{range .items}}{{.metadata.name}}{
{end}}')
zgg@LAPTOP-L21QMK1J:~$ echo "Name of the Pod: $POD_NAME"
Name of the Pod: kubernetes-bootcamp-9bc58d867-7lnjp
zgg@LAPTOP-L21QMK1J:~$ kubectl label pods $POD_NAME version=v1
kubectl get pods -l version=v1
pod/kubernetes-bootcamp-9bc58d867-7lnjp labeled
NAME                                   READY   STATUS    RESTARTS   AGE
kubernetes-bootcamp-9bc58d867-7lnjp    1/1     Running   0          10m
```

Figure 5.5: Resource insufficiency during scaling

b) Image Pull Delays: Multiple pods attempting to pull the same image simultaneously caused registry rate limiting and slow image downloads, resulting in prolonged ContainerCreating states.

Figure 5.6 shows the image pull delay issue.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl delete service kubernetes-bootcamp
service "kubernetes-bootcamp" deleted
zgg@LAPTOP-L21QMK1J:~$ kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   93m
zgg@LAPTOP-L21QMK1J:~$ curl http://127.0.0.1:38375
curl http://127.0.0.1:38375^C
zgg@LAPTOP-L21QMK1J:~$ kubectl exec -ti $POD_NAME -- curl http://localhost:8080
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-9bc58d867-7lnjp | v=1
```

Figure 5.6: Image pull delays during scaling

5.5 Troubleshooting and Resolution

To address the scaling issues, we implemented the following strategies:

a) Resource Management: We monitored cluster resource usage using kubectl top nodes and kubectl top pods to understand resource constraints. We adjusted resource requests and limits in the deployment specification to better match available cluster resources.

b) Image Optimization: To mitigate image pull delays, we considered pre-pulling images or using a local image registry to reduce download time and avoid rate limiting.

c) Gradual Scaling: We implemented a gradual scaling approach, scaling up in smaller increments to allow the cluster to adjust and avoid overwhelming resources.

The experiment demonstrated that scaling is not instantaneous and requires careful consideration of:

a) Available cluster resources

b) Image pull performance and registry limitations

c) Pod startup time and application initialization

d) Network and storage constraints

5.6 Load Balancing Verification

After successful scaling, we verified that traffic was being distributed across all pod instances by making multiple requests to the service endpoint. Each request was handled by a different pod, demonstrating the load balancing capabilities of Kubernetes services.

5.7 Conclusion

This experiment successfully demonstrated application scaling in Kubernetes. We scaled a deployment from 1 to 4 replicas, observed the scaling process, and resolved resource-related issues. The experiment highlighted the importance of understanding resource constraints and implementing appropriate scaling strategies. Scaling is a fundamental capability that enables applications to handle varying loads and provides fault tolerance through multiple instances.

Experiment 6: Rolling Update

6.1 Objectives

The objectives of this experiment are to:

a) Understand rolling updates in Kubernetes

b) Learn how to update application deployments

c) Observe the rolling update process

d) Learn how to rollback failed updates

6.2 Background Theory

Rolling updates allow Kubernetes to update applications with zero downtime by incrementally replacing old pods with new ones. Key characteristics of rolling updates include:

a) Gradual replacement: Pods are replaced one at a time or in small batches

b) Zero downtime: The application remains available throughout the update process

c) Automatic rollback: If the update fails, Kubernetes can automatically rollback to the previous version

d) Version control: Each update is versioned, allowing rollback to any previous version

6.3 Experimental Procedure

6.3.1 Current State Assessment

We first examined the current deployment and pod state:

kubectl get deployments

kubectl get pods

kubectl describe pods

These commands showed that we were running version 1 of the application with 4 replicas.

Figure 6.1 shows the initial deployment state.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl get deployments
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
kubernetes-bootcamp   1/1     1            1           18m
zgg@LAPTOP-L21QMK1J:~$ kubectl get rs
NAME                            DESIRED   CURRENT   READY   AGE
kubernetes-bootcamp-9bc58d867   1         1         1       18m
zgg@LAPTOP-L21QMK1J:~$ kubectl scale deployments/kubernetes-bootcamp --replicas=4
deployment.apps/kubernetes-bootcamp scaled
zgg@LAPTOP-L21QMK1J:~$ kubectl get deployments
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
kubernetes-bootcamp   4/4     4            4           19m
zgg@LAPTOP-L21QMK1J:~$ kubectl get pods -o wide
NAME                                   READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READIN
ESS GATES
kubernetes-bootcamp-9bc58d867-2tvzz   1/1     Running   0          34s   10.244.0.7   minikube   <none>           <none>
kubernetes-bootcamp-9bc58d867-7lnjp   1/1     Running   0          19m   10.244.0.5   minikube   <none>           <none>
kubernetes-bootcamp-9bc58d867-ljgvn   1/1     Running   0          34s   10.244.0.6   minikube   <none>           <none>
kubernetes-bootcamp-9bc58d867-wz85j   1/1     Running   0          34s   10.244.0.8   minikube   <none>           <none>
```

Figure 6.1: Initial deployment state with version 1

6.3.2 Performing Rolling Update

We initiated a rolling update to version 2 using the command:

> ***kubectl set image deployments/kubernetes-bootcamp kubernetes-bootcamp=docker.io/jocatalin/kubernetes-bootcamp:v2***

This command triggered the rolling update process, where Kubernetes gradually replaced the old pods with new pods running version 2 of the application.

Figure 6.2 shows the rolling update initiation.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl describe deployments/kubernetes-bootcamp
Name:                   kubernetes-bootcamp
Namespace:              default
CreationTimestamp:      Sun, 26 Apr 2026 13:56:06 +0800
Labels:                 app=kubernetes-bootcamp
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=kubernetes-bootcamp
Replicas:               4 desired | 4 updated | 4 total | 4 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=kubernetes-bootcamp
  Containers:
   kubernetes-bootcamp:
    Image:         gcr.io/google-samples/kubernetes-bootcamp:v1
    Port:          <none>
    Host Port:     <none>
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Progressing    True    NewReplicaSetAvailable
  Available      True    MinimumReplicasAvailable
OldReplicaSets:  <none>
NewReplicaSet:   kubernetes-bootcamp-9bc58d867 (4/4 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  19m   deployment-controller  Scaled up replica set kubernetes-bootcamp-9bc58d867 from 0 to
1
  Normal  ScalingReplicaSet  46s   deployment-controller  Scaled up replica set kubernetes-bootcamp-9bc58d867 from 1 to
4
```

Figure 6.2: Initiating rolling update to version 2

6.3.3 Monitoring Update Progress

We monitored the rolling update progress using:

kubectl get pods

kubectl rollout status deployments/kubernetes-bootcamp

We observed the gradual termination of old pods and creation of new pods. The rollout status command provided real-time information about the update progress.

Figure 6.3 shows the rolling update progress.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl describe services/kubernetes-bootcamp
Name:                     kubernetes-bootcamp
Namespace:                default
Labels:                   app=kubernetes-bootcamp
Annotations:              <none>
Selector:                 app=kubernetes-bootcamp
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.103.148.225
IPs:                      10.103.148.225
Port:                     <unset>  8080/TCP
TargetPort:               8080/TCP
NodePort:                 <unset>  32750/TCP
Endpoints:                10.244.0.5:8080,10.244.0.6:8080,10.244.0.7:8080 + 1 more...
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
zgg@LAPTOP-L21QMK1J:~$ curl http://127.0.0.1:41879
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-9bc58d867-7lnjp | v=1
zgg@LAPTOP-L21QMK1J:~$ curl http://127.0.0.1:41879
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-9bc58d867-2tvzz | v=1
zgg@LAPTOP-L21QMK1J:~$ curl http://127.0.0.1:41879
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-9bc58d867-2tvzz | v=1
zgg@LAPTOP-L21QMK1J:~$ curl http://127.0.0.1:41879
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-9bc58d867-2tvzz | v=1
```

Figure 6.3: Monitoring rolling update progress

6.3.4 Verifying Update Success

After the update completed, we verified that all pods were running version 2:

kubectl get pods

kubectl describe pods

We also accessed the application to confirm that it was serving the new version.

Figure 6.4 shows the successful update with version 2 pods.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl scale deployments/kubernetes-bootcamp --replicas=2
deployment.apps/kubernetes-bootcamp scaled
zgg@LAPTOP-L21QMK1J:~$ kubectl get deployments
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
kubernetes-bootcamp   2/2     2            2           31m
zgg@LAPTOP-L21QMK1J:~$ kubectl get pods -o wide
NAME                                   READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READIN
ESS GATES
kubernetes-bootcamp-9bc58d867-7lnjp    1/1     Running   0          32m   10.244.0.5   minikube   <none>           <none>
kubernetes-bootcamp-9bc58d867-wz85j    1/1     Running   0          13m   10.244.0.8   minikube   <none>           <none>
```

Figure 6.4: Successful update with version 2 pods

6.4 Error Analysis - Failed Update

To demonstrate error handling, we attempted to update to a non-existent version (v10):

> ***kubectl set image deployments/kubernetes-bootcamp kubernetes-bootcamp=gcr.io/google-samples/kubernetes-bootcamp:v10***

This update failed because the v10 image does not exist in the container registry.

Figure 6.5 shows the failed update attempt.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl get deployments
NAME                   READY   UP-TO-DATE   AVAILABLE   AGE
kubernetes-bootcamp    2/2     2            2           41m
zgg@LAPTOP-L21QMK1J:~$ kubectl get pods
NAME                                   READY   STATUS    RESTARTS   AGE
kubernetes-bootcamp-9bc58d867-7lnjp    1/1     Running   0          41m
kubernetes-bootcamp-9bc58d867-wz85j    1/1     Running   0          22m
zgg@LAPTOP-L21QMK1J:~$ kubectl describe pods
Name:             kubernetes-bootcamp-9bc58d867-7lnjp
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Sun, 26 Apr 2026 13:56:06 +0800
Labels:           app=kubernetes-bootcamp
                  pod-template-hash=9bc58d867
                  version=v1
Annotations:      <none>
Status:           Running
IP:               10.244.0.5
IPs:
  IP:           10.244.0.5
Controlled By:  ReplicaSet/kubernetes-bootcamp-9bc58d867
Containers:
  kubernetes-bootcamp:
    Container ID:   docker://fc02bf34ee3c8b5944ea03092e5e8c4953ca5e038d154e8dcc7f1fe3caa69781
    Image:          gcr.io/google-samples/kubernetes-bootcamp:v1
    Image ID:       docker-pullable://gcr.io/google-samples/kubernetes-bootcamp@sha256:0d6b8ee63bb57c5f5b6156f446b3bc3b3
c143d233037f3a2f00e279c8fcc64af
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Sun, 26 Apr 2026 13:57:07 +0800
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
```

Figure 6.5: Failed update to non-existent version v10

6.5 Error Investigation

We investigated the failure using:

> ***kubectl get deployments***
>
> ***kubectl get pods***
>
> ***kubectl describe pods***

The investigation revealed:

a) Deployment showed unavailable pods

b) Pods were in ImagePullBackOff or ErrImagePull state

c) Pod events indicated that the image could not be found

Figure 6.6 shows the pod status during the failed update.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl set image deployments/kubernetes-bootcamp kubernetes-bootcamp=docker.io/jocatalin/kuberne
tes-bootcamp:v2
deployment.apps/kubernetes-bootcamp image updated
zgg@LAPTOP-L21QMK1J:~$ kubectl get pods
NAME                                   READY   STATUS              RESTARTS   AGE
kubernetes-bootcamp-5c4f7cb664-lhjlr   0/1     ContainerCreating   0          12s
kubernetes-bootcamp-9bc58d867-7lnjp    1/1     Running             0          43m
kubernetes-bootcamp-9bc58d867-wz85j    1/1     Running             0          24m
zgg@LAPTOP-L21QMK1J:~$ kubectl expose deployment/kubernetes-bootcamp --type="NodePort" --port 8080
Error from server (AlreadyExists): services "kubernetes-bootcamp" already exists
zgg@LAPTOP-L21QMK1J:~$ kubectl rollout status deployments/kubernetes-bootcamp
deployment "kubernetes-bootcamp" successfully rolled out
zgg@LAPTOP-L21QMK1J:~$ kubectl describe pods
Name:             kubernetes-bootcamp-5c4f7cb664-lhjlr
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Sun, 26 Apr 2026 14:39:02 +0800
Labels:           app=kubernetes-bootcamp
                  pod-template-hash=5c4f7cb664
Annotations:      <none>
Status:           Running
IP:               10.244.0.9
IPs:
  IP:           10.244.0.9
```

Figure 6.6: Pod status during failed update

6.6 Rollback Procedure

We rolled back to the previous working version using:

kubectl rollout undo deployments/kubernetes-bootcamp

This command reverted the deployment to the previous stable version (v2).

Figure 6.7 shows the rollback command execution.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl describe services/kubernetes-bootcamp
Name:                     kubernetes-bootcamp
Namespace:                default
Labels:                   app=kubernetes-bootcamp
Annotations:              <none>
Selector:                 app=kubernetes-bootcamp
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.103.148.225
IPs:                      10.103.148.225
Port:                     <unset>  8080/TCP
TargetPort:               8080/TCP
NodePort:                 <unset>  32750/TCP
Endpoints:                10.244.0.9:8080,10.244.0.10:8080
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
zgg@LAPTOP-L21QMK1J:~$ export NODE_PORT=$(kubectl get services/kubernetes-bootcamp -o go-template='{{(index .spec.ports
0).nodePort}}')
zgg@LAPTOP-L21QMK1J:~$ echo NODE_PORT=$NODE_PORT
NODE_PORT=32750
```

Figure 6.7: Rolling back to previous version

6.7 Verifying Rollback Success

We verified that the rollback was successful by checking pod status and accessing the application:

kubectl get pods

kubectl describe pods

All pods were running version 2 again, and the application was functioning normally.

Figure 6.8 shows the successful rollback.

```
zgg@LAPTOP-L21QMK1J:~$ curl http://127.0.0.1:40421
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-5c4f7cb664-lhjlr | v=2
zgg@LAPTOP-L21QMK1J:~$ kubectl set image deployments/kubernetes-bootcamp kubernetes-bootcamp=gcr.io/google-samples/kuber
netes-bootcamp:v10
deployment.apps/kubernetes-bootcamp image updated
zgg@LAPTOP-L21QMK1J:~$ kubectl get pods
NAME                                   READY   STATUS         RESTARTS   AGE
kubernetes-bootcamp-5c4f7cb664-lhjlr   1/1     Running        0          11m
kubernetes-bootcamp-5c4f7cb664-nnh2s   1/1     Running        0          11m
kubernetes-bootcamp-75bd5fd495-bmszr   0/1     ErrImagePull   0          8s
zgg@LAPTOP-L21QMK1J:~$ kubectl rollout undo deployments/kubernetes-bootcamp
deployment.apps/kubernetes-bootcamp rolled back
zgg@LAPTOP-L21QMK1J:~$ kubectl get pods
NAME                                   READY   STATUS    RESTARTS   AGE
kubernetes-bootcamp-5c4f7cb664-lhjlr   1/1     Running   0          12m
kubernetes-bootcamp-5c4f7cb664-nnh2s   1/1     Running   0          11m
zgg@LAPTOP-L21QMK1J:~$ kubectl describe pods
Name:             kubernetes-bootcamp-5c4f7cb664-lhjlr
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Sun, 26 Apr 2026 14:39:02 +0800
Labels:           app=kubernetes-bootcamp
                  pod-template-hash=5c4f7cb664
Annotations:      <none>
Status:           Running
IP:               10.244.0.9
```

Figure 6.8: Successful rollback to version 2

6.8 Cleanup

Finally, we cleaned up the resources created during the experiment:

kubectl delete deployments/kubernetes-bootcamp services/kubernetes-bootcamp

Figure 6.9 shows the cleanup process.

```
zgg@LAPTOP-L21QMK1J:~$ kubectl delete deployments/kubernetes-bootcamp services/kubernetes-bootcamp
deployment.apps "kubernetes-bootcamp" deleted
service "kubernetes-bootcamp" deleted
```

Figure 6.9: Cleaning up deployment and service

6.9 Conclusion

This experiment successfully demonstrated rolling updates and rollback in Kubernetes. We performed a successful update from version 1 to version 2, observed the gradual replacement of pods, and verified that the application remained available throughout the process. We also intentionally caused a failed update and demonstrated the rollback procedure to restore the application to a working state. The experiment highlighted the importance of proper image versioning, thorough testing before deployment, and having

a rollback strategy in place. Rolling updates are a critical capability for maintaining application availability while deploying new features and bug fixes.

Conclusion

This laboratory experiment provided comprehensive hands-on experience with Kubernetes fundamentals. Through six detailed experiments, we explored the core concepts and operations of Kubernetes, including cluster creation, application deployment, pod and node management, service exposure, application scaling, and rolling updates.

Key learnings from this experiment include:

a) Cluster Management: We learned how to create and manage local Kubernetes clusters using Minikube, understanding the components and architecture of a Kubernetes cluster.

b) Application Deployment: We mastered the process of deploying applications to Kubernetes using Deployments, understanding the relationship between Deployments, ReplicaSets, and Pods.

c) Resource Monitoring: We developed skills in monitoring and troubleshooting Kubernetes resources using kubectl commands, understanding pod lifecycle states and node architecture.

d) Service Networking: We learned how to expose applications using Kubernetes Services, understanding service types, service discovery, and load balancing.

e) Application Scaling: We experienced horizontal scaling in Kubernetes, understanding resource constraints and the importance of proper resource management.

f) Application Updates: We mastered rolling updates and rollback procedures, understanding how to update applications with zero downtime and recover from failed updates.

The experiments were conducted using both Windows Subsystem for Linux (WSL) and Windows Command Prompt (CMD) environments, providing experience with different shell environments and command syntax. The troubleshooting experience gained from resolving various errors during the experiments provided valuable insights into common Kubernetes issues and their resolution strategies.

Overall, this laboratory experiment provided a solid foundation in Kubernetes fundamentals and practical experience with the platform. The skills and knowledge

gained from these experiments are directly applicable to real-world Kubernetes deployments and provide a strong basis for further exploration of advanced Kubernetes concepts and features.